

# ΑΝΑΓΝΩΡΙΣΗ ΠΡΟΤΥΠΩΝ

## Τελική Εργασία Εξαμήνου

Υπεύθυνος Καθηγητής: Ηλίας Θεοδωρακόπουλος

---

Δημήτριος Δούλαλας - ΑΕΜ: 58194

---



**ΔΗΜΟΚΡΙΤΕΙΟ  
ΠΑΝΕΠΙΣΤΗΜΙΟ  
ΘΡΑΚΗΣ**

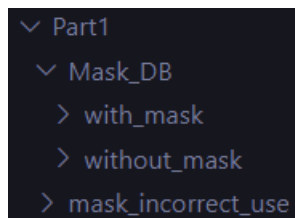
**DEMOCRITUS  
UNIVERSITY  
OF THRACE**

ΞΑΝΘΗ

Ιανουάριος 2024

## Μέρος 1<sup>ο</sup>

Αρχικά χώρισα τις εικόνες σε φακέλους όπως φαίνεται παρακάτω. Έτσι θα μπορώ να φορτώσω κατάλληλα τα δεδομένα μου αργότερα.



Επέλεξα να υλοποιήσω και να εκπαιδεύσω το σύστημά με ένα συνελκτικό νευρωνικό δίκτυο (CNN) για τους παρακάτω λόγους:

- Ικανότητα εξαγωγής χαρακτηριστικών: Τα CNNs έχουν την ικανότητα να εξάγουν αυτόματα χαρακτηριστικά από τις εικόνες. Τα convolutional layers εφαρμόζουν φίλτρα πάνω στην εικόνα, εξάγοντας διάφορα χαρακτηριστικά όπως ακμές, γωνίες, χρωματικά πρότυπα κ.α.
- Διαχείριση χωρικής ιεραρχίας: Τα CNNs μπορούν να αναγνωρίσουν ιεραρχικές δομές στις εικόνες (συνδυασμοί χαρακτηριστικών σε διάφορα επίπεδα) και κατά αυτόν τον τρόπο να αντιληφθούν πιο σύνθετα σχήματα και αντικείμενα.
- Pooling: Τα pooling layers επιτρέπουν τη μείωση διαστάσεων των χαρακτηριστικών, διατηρώντας συγχρόνως τα πιο σημαντικά χαρακτηριστικά, μειώνοντας έτσι την πολυπλοκότητα κι αυξάνοντας την αποδοτικότητα.

Δοκίμασα να υλοποιήσω μια αρχιτεκτονική παρόμοια με αυτή του AlexNet. Το πρώτο συνελκτικό επίπεδο είναι με 5x5 kernel και τα επόμενα είναι με 3x3 kernel. Παρέλειψα το πρώτο συνελκτικό επίπεδο που υπάρχει στο AlexNet με τον 11x11 kernel, γιατί οι εικόνες μας έχουν πολύ μικρές διαστάσεις (32x32), οπότε και δεν έχει νόημα να χρησιμοποιήσω τόσο μεγάλο kernel. Επίσης η συνάρτηση ενεργοποίησης στο τελευταίο επίπεδο είναι η Sigmoid (αντί της softmax που χρησιμοποιεί το (AlexNet), καθώς είναι η κατάλληλη για την δυαδική ταξινόμηση που θέλουμε να κάνουμε.

Η αρχιτεκτονική που υλοποίησα φαίνεται παρακάτω:

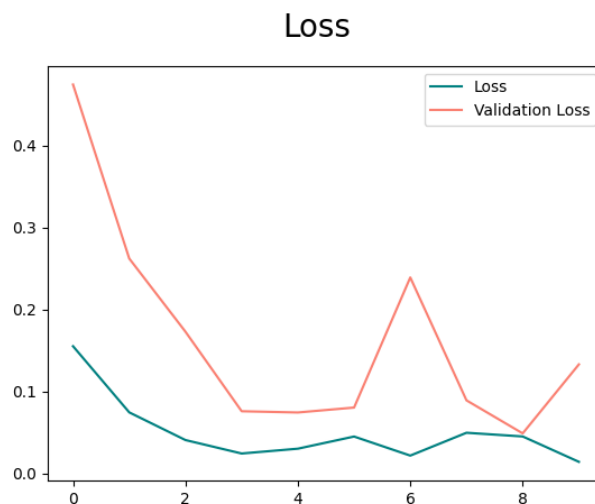
Model: "model"

| Layer (type)                              | Output Shape        | Param # | Connected to                                                                        |
|-------------------------------------------|---------------------|---------|-------------------------------------------------------------------------------------|
| input_1 (InputLayer)                      | [(None, 32, 32, 3)] | 0       | []                                                                                  |
| conv2d (Conv2D)                           | (None, 32, 32, 16)  | 1216    | ['input_1[0][0]']                                                                   |
| batch_normalization (Batch Normalization) | (None, 32, 32, 16)  | 64      | ['conv2d[0][0]']                                                                    |
| max_pooling2d (MaxPooling2D)              | multiple            | 0       | ['batch_normalization[0][0]', 'conv2d_1[0][0]', 'conv2d_2[0][0]', 'conv2d_3[0][0]'] |
| conv2d_1 (Conv2D)                         | (None, 16, 16, 32)  | 4640    | ['max_pooling2d[0][0]']                                                             |
| conv2d_2 (Conv2D)                         | (None, 8, 8, 64)    | 18496   | ['max_pooling2d[1][0]']                                                             |
| conv2d_3 (Conv2D)                         | (None, 4, 4, 128)   | 73856   | ['max_pooling2d[2][0]']                                                             |
| flatten (Flatten)                         | (None, 512)         | 0       | ['max_pooling2d[3][0]']                                                             |
| dense (Dense)                             | (None, 128)         | 65664   | ['flatten[0][0]']                                                                   |
| dropout (Dropout)                         | (None, 128)         | 0       | ['dense[0][0]']                                                                     |
| dense_1 (Dense)                           | (None, 1)           | 129     | ['dropout[0][0]']                                                                   |

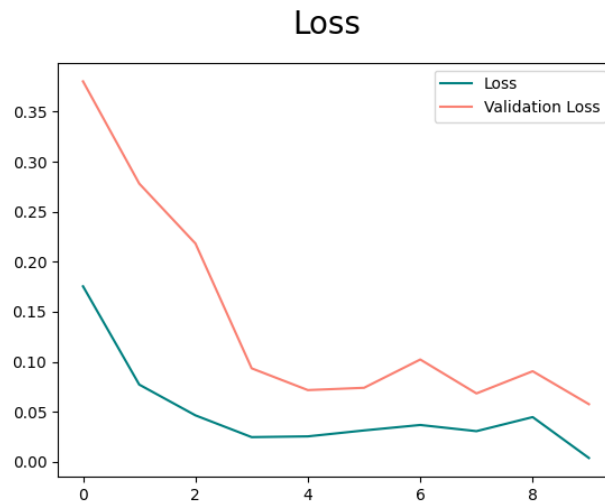
=====  
 Total params: 164065 (640.88 KB)  
 Trainable params: 164033 (640.75 KB)  
 Non-trainable params: 32 (128.00 Byte)

Δοκίμασα 2 παραλλαγές: α) με 3 συνελκτικά επίπεδα (αφαιρώντας ένα 3x3 συνελκτικό επίπεδο) και β) με 4 συνελκτικά επίπεδα (αυτή που φαίνεται παραπάνω).

Οι επιδόσεις τους στο validation\_set φαίνονται στα παρακάτω διαγράμματα που απεικονίζουν την εξέλιξη του Validation Loss ανά κάθε εποχή.



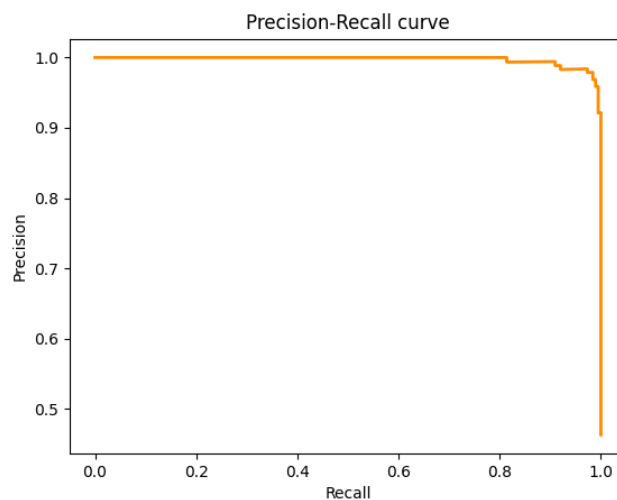
α) με 3 συνελκτικά επίπεδα



### β) με 4 συνελκτικά επίπεδα

Για το δεύτερο μοντέλο - το οποίο είναι το καλύτερο – υπολόγισα το σφάλμα ταξινόμησης στο σύνολο δοκιμής, καθώς και τις καμπύλες precision-recall και το AUC, όπως φαίνονται παρακάτω:

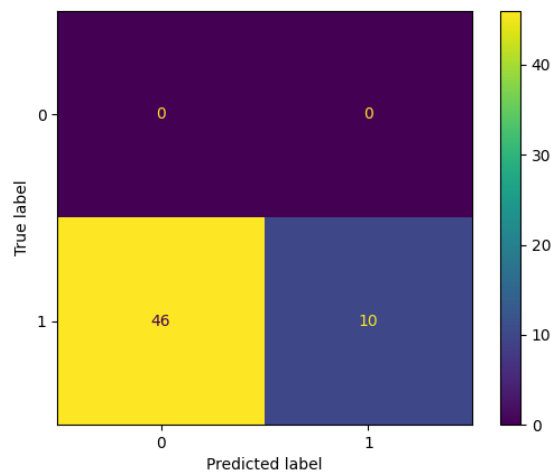
**Test Loss: 0.0794651210308075, Test Accuracy: 0.9802955389022827**



Precision-Recall διάγραμμα

**AUC: 0.9973778624066936**

Τέλος, εφαρμόζοντας το εκπαιδευμένο μοντέλο στις εικόνες της κλάσης `mask_incorrect_use` υπολόγισα το ποσοστό αυτών που ταξινομούνται εσφαλμένα ως αποδεκτή χρήση μάσκας. Για να οπτικοποιηθεί το αποτέλεσμα χρησιμοποίησα τον πίνακα σύγχυσης όπως φαίνεται παρακάτω:



Παρατηρώ πως το ποσοστό αυτών που ταξινομούνται εσφαλμένα ως αποδεκτή χρήση μάσκας είναι πολύ μεγάλο ( $46/(46+10) = 82,14\%$ ).

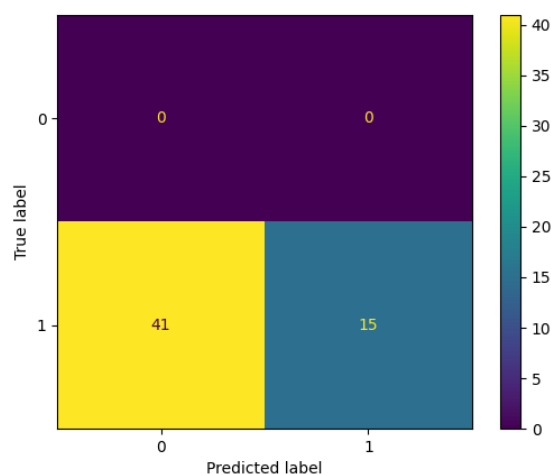
Ένας τρόπος να μειωθεί αυτό το ποσοστό, χρησιμοποιώντας μόνο τα υπάρχοντα δεδομένα, είναι μειώνοντας το κατώφλι ως εξής:

```

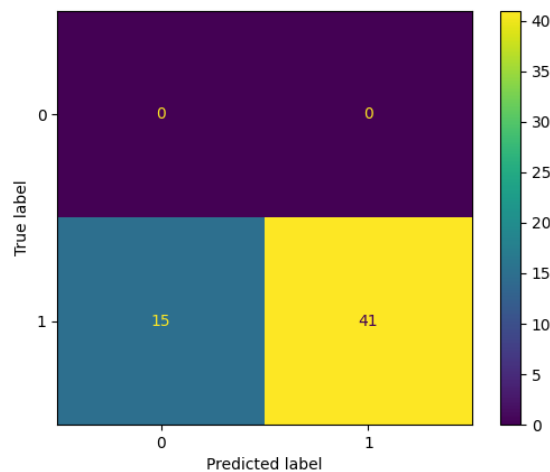
yhat = model.predict(batch)
for item in yhat:
    if item >= 0.5:
        item = 1
    if item < 0.5:
        item = 0
    preds_inc_use.append(item)

```

Κάνοντάς το 0.0001 από 0.5 παίρνω τα παρακάτω αποτελέσματα:



Το ποσοστό μειώνεται σε  $73,21\%$ . Μειώνοντας το κι άλλο σε 0.00001 παίρνω τα παρακάτω αποτελέσματα:



Το ποσοστό πηγαίνει στο **26,79%**. Βέβαια με τόσο χαμηλό κατώφλι επηρεάζονται πλέον σημαντικά και τα αποτελέσματα του `test_set`, οπότε ίσως ιδανικότερη τιμή είναι η προηγούμενη.

## Μέρος 2<sup>ο</sup>

Όπως και στο 1<sup>ο</sup> μέρος, επέλεξα ξανά νευρωνικό αλγόριθμο, διότι τα νευρωνικά δίκτυα μπορούν να εκπαιδευτούν για ανάληψη πολύπλοκων και μη-γραμμικών συσχετίσεων μεταξύ των δεδομένων, καθώς και να αντιμετωπίσουν την μεγάλη διαστασιμότητα (high dimensionality) στα δεδομένα, όπου οι παραδοσιακοί συμβατικοί αλγόριθμοι ενδέχεται να αντιμετωπίσουν ίσως περισσότερα προβλήματα.

Πιο συγκεκριμένα το μοντέλο έχει:

1. Επίπεδα συνεχών και δυαδικών χαρακτηριστικών: 2 σετ επιπέδων: α) 1 σετ επιπέδων για συνεχή χαρακτηριστικά και β) 1 σετ επιπέδων για δυαδικά χαρακτηριστικά. Στα επίπεδα αυτά εφαρμόζονται γραμμικοί μετασχηματισμοί στα εισερχόμενα δεδομένα με συνάρτηση ενεργοποίησης την ReLU.
2. Συνδυαστικά επίπεδα: επίπεδα που λαμβάνουν τα αποτελέσματα από τα παραπάνω επίπεδα και εφαρμόζουν κι άλλους γραμμικούς μετασχηματισμούς με συνάρτηση ενεργοποίησης την ReLU.
3. Επίπεδο εξόδου: είναι το τελευταίο επίπεδο. Εφαρμόζει έναν γραμμικό μετασχηματισμό προκειμένου να παράγει την τελική έξοδο του μοντέλου με χρήση σιγμοειδούς συνάρτησης ενεργοποίησης.

Δεν χρησιμοποιήθηκε καθόλου dropout επίπεδο (σε αντίθεση με το 1<sup>ο</sup> μέρος), καθώς το συγκεκριμένο πρόβλημα δεν μπορεί να οδηγηθεί εύκολα σε overfitting.

Οι παραλλαγές που δοκίμασα στο μοντέλο είναι τρεις:

1. 2 συνδυαστικά επίπεδα με 3 επίπεδα για συνεχή χαρακτηριστικά, 3 επίπεδα για δυαδικά χαρακτηριστικά και ένα επίπεδο εξόδου.
2. 2 συνδυαστικά επίπεδα με 5 επίπεδα για συνεχή χαρακτηριστικά, 5 επίπεδα για δυαδικά χαρακτηριστικά και ένα επίπεδο εξόδου.
3. 3 συνδυαστικά επίπεδα με 5 επίπεδα για συνεχή χαρακτηριστικά, 5 επίπεδα για δυαδικά χαρακτηριστικά και ένα επίπεδο εξόδου.

Οι επιδόσεις τους στο validation\_set φαίνονται παρακάτω:

```
(env3.11.7) PS C:\Python_repo\Final_Project\Part2> python main2.py
Epoch 1/10, Loss: 53.57143020629883
Epoch 2/10, Loss: 53.57143020629883
Epoch 3/10, Loss: 28.571428298950195
Epoch 4/10, Loss: 46.42856979370117
Epoch 5/10, Loss: 28.571428298950195
Epoch 6/10, Loss: 46.42856979370117
Epoch 7/10, Loss: 35.71428680419922
Epoch 8/10, Loss: 35.71428680419922
Epoch 9/10, Loss: 32.14285659790039
Epoch 10/10, Loss: 35.71428680419922
Validation accuracy: 72.6457399103139%
```

1.

```
(env3.11.7) PS C:\Python_repo\Final_Project\Part2> python main2.py
Epoch 1/10, Loss: 0.5097588896751404
Epoch 2/10, Loss: 0.33641842007637024
Epoch 3/10, Loss: 0.536646842956543
Epoch 4/10, Loss: 0.23422767221927643
Epoch 5/10, Loss: 0.11787150800228119
Epoch 6/10, Loss: 0.0806279182434082
Epoch 7/10, Loss: 0.34614232182502747
Epoch 8/10, Loss: 0.08355533331632614
Epoch 9/10, Loss: 0.20919403433799744
Epoch 10/10, Loss: 0.12688077986240387
Validation accuracy: 86.99551569506727%
```

2.

```
(env3.11.7) PS C:\Python_repo\Final_Project\Part2> python main.py
Epoch 1/10, Loss: 0.7709977030754089
Epoch 2/10, Loss: 0.4909980297088623
Epoch 3/10, Loss: 0.7481823563575745
Epoch 4/10, Loss: 0.16483250260353088
Epoch 5/10, Loss: 0.3947695791721344
Epoch 6/10, Loss: 0.2852078676223755
Epoch 7/10, Loss: 0.2522878348827362
Epoch 8/10, Loss: 0.4048231542110443
Epoch 9/10, Loss: 0.28070884943008423
Epoch 10/10, Loss: 0.22141008079051971
Validation accuracy: 90.13452914798206%
```

3.

Με βάση το Validation accuracy, το καλύτερο μοντέλο είναι το 3<sup>ο</sup>.